

Récap — BDD sécurisée, Dictionnaire & Professeur Sakura complet

Migration de LocalStorage vers une vraie base de données cross-device, verrouillage sécurité, nouvelle section Dictionnaire, et l'agent IA Professeur Sakura devenu pleinement utilisable — génération ET entraînement, pour les flashcards comme pour les QCM.

2 septembre 2026

AUDIT.md → Tâches 6, 13, 13-bis, 14, 14-bis, 15, 16, 17

TÂCHE 16

✓ Terminée

BDD Supabase — remplace LocalStorage comme source de vérité

Le site publié sur Netlify a rendu visible une limite connue : LocalStorage ne synchronise rien entre appareils ou navigateurs. Fiches, flashcards et QCM sont désormais persistés dans une vraie base de données (Supabase / Postgres), avec repli automatique sur LocalStorage si la base est injoignable — jamais de blocage.

Réalisé

- Schéma générique volontaire — 3 tables (`fiches` , `flashcards` , `qcm`), chacune `{id, data jsonb, updated_at}` — `data` porte exactement le même objet JS qu'avant en LocalStorage, aucune migration de colonnes à refaire si le modèle évolue.
- `useSupabaseStore.js` remplace `useLocalStorage` : même signature, écriture en diff (upsert/delete) non bloquante, cache local systématique en parallèle.
- Premier lancement sur une base vide : le cache/mock local est poussé vers Supabase plutôt que d'effacer les données déjà à l'écran.

VÉRIFIÉ EN CONDITIONS RÉELLES

Fiche modifiée depuis l'UI → retrouvée à l'identique en base par requête directe. Tables absentes (avant exécution du script SQL) → avertissement console clair, app fonctionnelle sur le cache local, aucun blocage.

Verrouillage sécurité — plus aucune clé Supabase côté navigateur

La clé `anon` de la Tâche 16, bien que publique par design, donnait un accès direct en lecture/écriture aux 3 tables depuis n'importe où (visible dans le bundle JS). Objectif : qu'aucune clé, publique ou secrète, ne soit accessible depuis le navigateur.

Réalisé

- Passerelle serveur `netlify/functions/data.js` — seule à détenir la clé `service_role` (secrète), stockée en variable d'environnement sans préfixe `VITE_` donc jamais inlinée dans le bundle par Vite.
- Le front ne parle plus qu'à cette fonction (`fetch` vers `/.netlify/functions/data`) ; politiques RLS Supabase publiques supprimées, RLS activée sans aucune policy → deny-all pour la clé anon.
- `netlify dev` ajouté comme configuration locale pour tester les fonctions serveur, en plus du Vite seul déjà existant.

VÉRIFIÉ EN CONDITIONS RÉELLES

`curl` direct sur l'API Supabase avec l'ancienne clé anon → `200` mais tableau vide en lecture, `401 row-level security policy` en écriture. Bundle JS de prod inspecté (URL et motif de clé JWT) → **0 occurrence**. Écriture bout en bout depuis l'UI confirmée en base.

Dictionnaire — vue agrégée des définitions

Le scope initial prévoyait un bouton dédié et un nouveau scénario Make. Recadrage plus simple, demandé en cours de route : une nouvelle section dans la Sidebar qui affiche les définitions **déjà extraites** par l'Écriture magique (Tâche 7) sur toutes les fiches — aucun nouveau webhook Make, aucune nouvelle action IA à valider.

Réalisé

- Nouvel item "Dictionnaire" dans la Sidebar, sous Coin Study.
- `DictionaryView.jsx` : pour chaque fiche filtrée (recherche/UE/Thème du Header, comme Coin Study) ayant des définitions, aplatit le tout en liste triée alphabétiquement — terme, définition, UE, fiche source.
- Clic sur une entrée → ouvre la fiche source dans la même modale que Coin Study.

Écriture magique cassée après une modification — diagnostiqué et corrigé

Le bouton "Écriture magique" renvoyait "Réponse inattendue du service IA". Diagnostic mené en lisant directement le blueprint du scénario Make via son API — la cause n'était pas visible au premier coup d'œil dans l'interface.

① Deux headers HTTP fusionnés en une seule entrée invalide.

Lors d'une modification précédente (retrait d'un module Parse JSON fragile), `Content-Type` et `Access-Control-Allow-Origin` s'étaient retrouvés collés dans un seul champ malformé (`"Content-Type Access-Control-Allow-Origin": "application/json *"`) — un nom de header avec un espace n'est pas valide HTTP.

② Correctif appliqué directement via l'API Make.

Headers splittés en 2 entrées propres ; `max_tokens` remonté de 1024 à 2048 par prudence (marge de sécurité sur les réponses longues).

RÉSULTAT

2 appels webhook de test directs → JSON propre et complet ; exécutions Make repassées en statut SUCCESS ; test réel dans l'app → 4 définitions extraites, visibles dans le Dictionnaire.

Professeur Sakura — Flashcards, génération & entraînement

L'app était prête côté code depuis un moment, mais le webhook pointait vers un scénario Make jamais réellement construit (`HTTP 410 - no scenario listening`). Plutôt qu'un 3^e scénario (le plan Make Free plafonne à 2 actifs), une branche Router a été ajoutée au scénario déjà actif de l'Écriture magique, dispatchée par un champ `type`.

Génération (Tâche 13)

- Router Make étendu : nouvelle branche `type = sakura-flashcards`, prompt dédié, réponse `{ decks: [...] }`.
- Écriture magique non modifiée (rétrocompatible via un filtre "type vide") — aucune régression.

Entraînement (Tâche 13-bis) — demande utilisateur

"Les decks c'est chouette mais il faut pouvoir cliquer dessus et les faire" — jusqu'ici la liste "Mes decks" était en lecture seule par décision de cadrage initiale.

- Nouveau composant `FlashcardTrainer.jsx` (nom déjà anticipé dans `CLAUDE.md`) : une carte à la fois, clic pour retourner (question ↔ réponse), Précédent/Suivant, barre de progression.
- Dernière carte → "Recommencer" à la place de "Suivant".
- Chaque deck de la liste "Mes decks" devient cliquable et ouvre directement l'entraînement.

VÉRIFIÉ EN CONDITIONS RÉELLES

Génération testée par `curl` et dans l'app (3 decks générés sur une UE entière) ; entraînement testé sur un deck existant (retournement, navigation, fin de deck).

Professeur Sakura — QCM, génération & correction question par question

Génération (Tâche 14)

- 2^e branche Router (`type = sakura-qcm`) : nombre de questions interpolé dans le prompt, `max_tokens` à 3000 (réponses plus longues qu'un deck).

Mode "passer le QCM" (Tâche 14-bis) — demande utilisateur

"Il faut que on clique dessus et qu'on puisse répondre aux questions et avoir la correction avec explication après chaque envoi de réponse."

- Nouveau composant `QcmTrainer.jsx` : une question à la fois, sélection d'une option, "Valider ma réponse".
- Correction immédiate après validation — bonne réponse toujours en vert (✓), réponse choisie en rouge (X) si fausse, explication affichée juste en dessous ; sélection verrouillée après validation.
- Écran final : score X / N, pourcentage, "Recommencer" (remise à zéro).

VÉRIFIÉ EN CONDITIONS RÉELLES

QCM de 3 questions joué en entier (réponse volontairement fausse puis correctes) → correction cohérente à chaque étape → score final exact affiché.

Professeur Sakura — Discuter (chat libre)

3^e et dernière branche du Router (`type = sakura-chat`), historique récent + nouveau message interpolés directement dans le prompt. Testé par `curl` puis en conditions réelles sur le site en production — réponse pédagogique cohérente reçue et affichée.

FONCTIONNALITÉ	ÉTAT
BDD Supabase (cross-device)	✓ En ligne
Verrouillage sécurité (clé service_role serveur)	✓ En ligne
Dictionnaire (agrégation des définitions)	✓ En ligne
Écriture magique	✓ En ligne
Sakura — Flashcards (génération + entraînement)	✓ En ligne
Sakura — QCM (génération + passage + correction)	✓ En ligne
Sakura — Discuter	✓ En ligne
Sync Google Sheets	✓ En ligne
Rappels quotidiens (Tâche 2)	• Bloqué — plan Make Free (2 scénarios actifs max)
Récap week-end / Sync Calendar (Tâches 4-5)	• Pas encore designées

Toutes les fonctionnalités IA tournent désormais sur un seul scénario Make actif (Router à 4 branches), sans dépasser la limite de 2 scénarios actifs du plan Free — le 2^e slot sert la sync Google Sheets. Tout déployé et vérifié sur **edustudy-hub.netlify.app**.

Généré à partir de la session de travail et des mises à jour d'AUDIT.md. Données fictives uniquement, aucune clé API en dur — conforme à la Charte IA & Sécurité du projet.